# RMI

## 1. Introduction to RMI

**RMI stands for Remote Method Invocation.**
Java RMI is a technology that allows an object running in one Java Virtual Machine (JVM) to **invoke methods on an object running in another JVM**, possibly on another computer.

In simple words:

> **RMI allows a Java program to call a method of a remote object just like calling a normal method.**

For example, a client can execute:

```
int result = obj.add(10, 20);
```

Although obj may actually be running on another machine.

## 2. Why Do We Need RMI?

Normally, when we call a method:

```
obj.add(10, 20);
```

both the object and the calling program are running in the same JVM.
With RMI:

```
Client JVM                        Server JVM
```

```
Client program                  Remote Object
     |                               |
     | -------- add(10,20) ------------>|
     |                               |
     | <----------- 30 -----------------|
```

The client does not need to know the internal implementation of the remote object.

## 3. RMI Architecture

The basic RMI architecture contains the following important components:

1. **Client**
2. **Server**
3. **Remote Interface**
4. **Remote Object**
5. **RMI Registry**
6. **Stub**
7. **Skeleton / Server-side dispatch mechanism**

### Basic Architecture

```
                  RMI Registry
```

```
            |
        lookup / bind
            |
            |
   +---------------+---------------+
   |                     |
   ↓                     ↓
 RMI Client              RMI Server
   |                     |
   |                     |
  Stub  ----------------------> Remote Object
   |                     |
   |                     |
   +------ Remote Method ----------+
```

## 4. Components of RMI Architecture

### A. Client

The **client** is the Java program that wants to call a method of a remote object.
Example:

```
MyInterface obj =
```

```
  (MyInterface) Naming.lookup("rmi://localhost/Add");

int result = obj.add(10, 20);
```

The client:

- contacts the RMI Registry
- finds the remote object
- obtains a reference to the remote object
- invokes its method

### B. Server

The **server** creates and registers the remote object.
Example:

```
MyRemote obj = new MyRemote();
```

```
Naming.rebind("rmi://localhost/Add", obj);
```

The server:

- creates the remote object
- exports the object for remote access
- registers the object with the RMI Registry
- waits for clients

## 5. Remote Interface

A **Remote Interface** defines the methods that can be called remotely.
It must extend:

```
java.rmi.Remote
```

Example:

```
import java.rmi.*;
```

```
public interface MyInterface extends Remote {

int add(int a, int b) throws RemoteException;
}
```

### Important points

A remote interface:

- must extend Remote
- contains methods that can be called remotely
- remote methods normally declare RemoteException

For example:

```
int add(int a, int b) throws RemoteException;
```

## 6. Remote Object

The remote object contains the actual implementation of the remote methods.
Example:

```
import java.rmi.*;
```

```
import java.rmi.server.*;

public class MyRemote extends UnicastRemoteObject
implements MyInterface {

MyRemote() throws RemoteException {
super();
   }

public int add(int a, int b) {
return a + b;
   }
}
```

Here:

```
implements MyInterface
```

means the class implements the remote interface.
And:

```
extends UnicastRemoteObject
```

makes the object available for RMI communication.

## 7. RMI Registry

The **RMI Registry** is a naming service.
It stores the name and reference of remote objects.
For example:

```
Naming.rebind("rmi://localhost/Add", obj);
```

Here:

```
Add
```

is the name given to the remote object.
The client later searches for the object using:

```
Naming.lookup("rmi://localhost/Add");
```

### Simple analogy

Think of the RMI Registry as a **telephone directory**.

```
Name        Object
```

```
------------------------
Add        MyRemote object
```

The client knows the name Add, searches the registry, and gets the remote reference.

## 8. Stub

A **Stub** is the client-side representation of the remote object.
The client does not directly communicate with the actual remote object.
Instead:

```
Client → Stub → Network → Remote Object
```

The Stub:

- receives the method call from the client
- converts/packages the method arguments
- sends the request to the remote JVM
- receives the response
- returns the result to the client

Modern Java RMI handles stub generation differently from older versions, so students generally **do not need to manually create a stub** for this basic experiment.

## 9. Skeleton

In older RMI architecture, the **Skeleton** was the server-side component responsible for receiving requests from the stub and invoking the appropriate method on the remote object.
Conceptually:

```
Client
```

```
  |
 Stub
  |
 Network
  |
Skeleton
  |
Remote Object
```

In modern Java RMI, the explicit skeleton mechanism used in older Java versions is no longer something students normally create manually. The RMI runtime handles the server-side dispatch.

### For exam purposes

You can write:

> **Skeleton is the server-side counterpart of the Stub that receives remote requests and invokes the corresponding method on the remote object. In modern Java RMI, explicit skeleton classes are handled internally by the RMI runtime.**

## 10. Complete RMI Communication

Suppose the client executes:

```
obj.add(10, 20);
```

The following process takes place:

```
Step 1
```

```
Client calls add(10,20)

     ↓

Step 2
Client Stub receives the call

     ↓

Step 3
Stub sends request through RMI

     ↓

Step 4
Server receives the request

     ↓

Step 5
Remote Object executes:

    add(10,20)

     ↓

Step 6
Remote Object returns 30

     ↓

Step 7
Result travels back to client

     ↓

Step 8
Client receives 30
```

## 11. RMI Architecture Diagram

Students can draw this diagram in their practical file:

```
                          RMI REGISTRY
```

```
              +---------------+
              |  Add        |
              |  Remote Obj  |
              +-------+-------+
                   |
              lookup / bind
                   |
       +--------------------+--------------------+
       |                              |
       ↓                              ↓

   CLIENT JVM                    SERVER JVM
+-------------------+              +-------------------+
| Client Program    |              | Remote Object     |
|               |              |               |
| obj.add(10,20)    |              | MyRemote          |
+---------+---------+              +---------+---------+
       |                              ^
       ↓                              |
     Stub  ----------- Network  ---------------+
       |
       ↓
    Result = 30
```

## 12. RMI Experiment

### Experiment Title

**Implementation of Remote Method Invocation using Java RMI**

### Aim

To develop a simple Java RMI application in which a client invokes a method of a remote object to perform addition of two numbers.

## 13. Software Requirements

- Java JDK
- Command Prompt / Terminal
- Text Editor or IDE
- Two or more Java source files
- Same network if client and server are on different computers

For the first demonstration, use:

```
localhost
```

This allows students to run both client and server on the same computer.

## 14. Files Required

Create the following four files in the same folder:

```
RMI
```

```
│
├──── MyInterface.java
├──── MyRemote.java
├──── Server.java
└──── Client.java
```

## 15. Program 1 — Remote Interface

### MyInterface.java

```
import java.rmi.Remote;
```

```
import java.rmi.RemoteException;

public interface MyInterface extends Remote {

int add(int a, int b) throws RemoteException;
}
```

### Explanation

```
import java.rmi.Remote;
```

Imports the Remote interface.

```
import java.rmi.RemoteException;
```

Imports the exception used for remote communication.

```
public interface MyInterface extends Remote
```

Defines the remote interface.

```
int add(int a, int b) throws RemoteException;
```

Declares the method that the client can invoke remotely.

## 16. Program 2 — Remote Implementation

### MyRemote.java

```
import java.rmi.RemoteException;
```

```
import java.rmi.server.UnicastRemoteObject;

public class MyRemote extends UnicastRemoteObject
implements MyInterface {

public MyRemote() throws RemoteException {
super();
   }

public int add(int a, int b) {
return a + b;
   }
}
```

### Explanation

```
extends UnicastRemoteObject
```

Exports the object so that it can receive remote calls.

```
implements MyInterface
```

Provides implementation of the remote interface.

```
public MyRemote() throws RemoteException
```

Constructor handles possible RMI-related exceptions.

```
public int add(int a, int b)
```

Actual implementation of the remote method.

## 17. Program 3 — Server

### Server.java

```
import java.rmi.Naming;
```

```
public class Server {

public static void main(String[] args) {

try {
MyRemote obj = new MyRemote();

Naming.rebind("rmi://localhost/Add", obj);

System.out.println("Server is ready...");
      }
catch (Exception e) {
System.out.println(e);
      }
   }
}
```

### Explanation

First:

```
MyRemote obj = new MyRemote();
```

creates the remote object.
Then:

```
Naming.rebind("rmi://localhost/Add", obj);
```

registers the object with the RMI Registry using the name:

```
Add
```

Finally:

```
System.out.println("Server is ready...");
```

displays the server status.

## 18. Program 4 — Client

### Client.java

```
import java.rmi.Naming;
```

```
public class Client {

public static void main(String[] args) {

try {

MyInterface obj =
           (MyInterface) Naming.lookup(
"rmi://localhost/Add");

int result = obj.add(10, 20);

System.out.println("Result = " + result);
      }
catch (Exception e) {
System.out.println(e);
      }
   }
}
```

### Explanation

The important statement is:

```
Naming.lookup("rmi://localhost/Add");
```

The client searches the RMI Registry for the object named:

```
Add
```

Then:

```
int result = obj.add(10, 20);
```

invokes the remote method.
The server performs:

```
return a + b;
```

and returns:

```
30
```

## 19. How to Execute the Experiment

Open Command Prompt in the project folder.

### Step 1 — Compile

```
javac *.java
```

If compilation is successful, .class files will be created.

### Step 2 — Start RMI Registry

Open another Command Prompt in the same folder:

```
start rmiregistry
```

The registry will start in the background.

### Step 3 — Start Server

Open another terminal:

```
java Server
```

Output:

```
Server is ready...
```

### Step 4 — Run Client

Open another terminal:

```
java Client
```

Output:

```
Result = 30
```

## 20. Complete Execution Flow

Students should understand this sequence:

```
              START
```

```
        |
        ↓
   Compile Java Programs
        |
        ↓
   Start RMI Registry
        |
        ↓
    Start RMI Server
        |
        ↓
  Server creates object
        |
        ↓
  Server registers object
        |
        ↓
    Start RMI Client
        |
        ↓
   Client performs lookup
        |
        ↓
   Client gets reference
        |
        ↓
    Client calls add()
        |
        ↓
   Remote object executes
        |
        ↓
      Result = 30
        |
        ↓
        END
```

## 21. Important RMI Methods

### Naming.rebind()

Used by the server to register a remote object.

```
Naming.rebind("rmi://localhost/Add", obj);
```

Meaning:

> Register obj with the name Add.

### Naming.lookup()

Used by the client to find the remote object.

```
Naming.lookup("rmi://localhost/Add");
```

Meaning:

> Find the object registered with the name Add.

## 22. Difference Between bind() and rebind()

### bind()

Registers an object.

```
Naming.bind("Add", obj);
```

If the name already exists, an exception can occur.

### rebind()

Registers or replaces an existing binding.

```
Naming.rebind("Add", obj);
```

For simple classroom experiments, rebind() is convenient.

## 23. What Happens When Client Calls add()?

Suppose:

```
obj.add(10, 20);
```

The client thinks it is calling a normal method.
Actually:

```
Client
```

```
  |
  | add(10,20)
  ↓
Stub / RMI Runtime
  |
  | Network request
  ↓
Server JVM
  |
  ↓
MyRemote.add()
  |
  ↓
10 + 20
  |
  ↓
30
  |
  ↓
Client
```

This is the central concept of RMI.

## 24. Important Classes and Interfaces

| Class / Interface | Purpose |
|---|---|
| Remote | Identifies a remote interface |
| RemoteException | Handles remote communication errors |
| UnicastRemoteObject | Supports exporting remote objects |
| Naming | Provides methods for finding/registering remote objects |
| RMI Registry | Stores remote object names and references |

## 25. RMI vs Normal Method Call

### Normal Method Call

```
Client/Object
```

```
  |
  ↓
Method
  |
  ↓
Result
```

Everything happens inside the same JVM/process.

### RMI Method Call

```
Client JVM
```

```
  |
  ↓
RMI Runtime
  |
  ↓
Network
  |
  ↓
Server JVM
  |
  ↓
Remote Object
  |
  ↓
Result
```

The object can be in another JVM or another computer.

## 26. RMI Advantages

1. **Easy remote communication**
2. Uses normal Java method-call syntax
3. Object-oriented approach
4. Supports distributed Java applications
5. Provides abstraction from network communication
6. Supports passing Java objects as parameters/return values when they are serializable or otherwise suitable for RMI

## 27. RMI Disadvantages

1. Mainly suitable for Java-to-Java communication.
2. Network failures can occur.
3. Remote calls are slower than local method calls.
4. Configuration can be more complicated than a normal Java application.
5. Client and server must agree on the remote interface.

## 28. Viva Questions

### Q1. What is RMI?

**Answer:** RMI stands for Remote Method Invocation. It allows a Java program to invoke methods on an object running in another JVM.

### Q2. What is the purpose of the RMI Registry?

**Answer:** It is a naming service used to register and locate remote objects.

### Q3. Which interface must a remote interface extend?

**Answer:**

```
java.rmi.Remote
```

### Q4. Why is RemoteException used?

**Answer:** Because remote communication can fail due to network or other communication-related problems.

### Q5. What is a Stub?

**Answer:** A Stub is the client-side representation/proxy of a remote object through which the client makes remote method calls.

### Q6. What is UnicastRemoteObject?

**Answer:** It is a class used to export a remote object so it can receive RMI calls.

### Q7. Which method is used to register an object?

**Answer:**

```
Naming.rebind()
```

### Q8. Which method is used by the client to obtain a remote reference?

**Answer:**

```
Naming.lookup()
```

### Q9. What is the difference between localhost and an IP address?

**Answer:**

```
localhost → current computer
```

192.168.x.x → another computer on the network

### Q10. What is the role of the remote interface?

**Answer:** It defines the methods that can be invoked remotely.